

Rain OS Commercial Build Specification

Document type: single integrated product, technical, architecture, operations, and implementation specification

Revision: 1.1 GitHub Actions parallel-build update

Generated: 2026-09-12

Public repository audited: <https://github.com/dasara-varun/rain-os>

Audited commit: [d41587f56bc09b3b9a81804477673264433213b0](#)

Primary build system: GitHub Actions parallel build pipeline with Arch Linux container/self-hosted runner options

Developer fallback: Windows ¹⁰/₁₁ with WSL2 and an official Arch Linux WSL distribution

Target artifact: bootable x86_64 Archiso ISO, followed by a tested commercial desktop release

Status: This document is an implementation plan and release specification. It does not claim that Rain OS is already a production-ready commercial operating system. The current public repository is a working prototype and must pass the gates defined here.

How to use this document

This specification is intentionally organized as a single document so that an AI-agent team can work through it in one controlled loop. Each numbered page is a decision surface. A page contains a requirement, the implementation contract, concrete files or packages, verification evidence, and risks. The `\newpage` markers are deliberate: rendering this Markdown with a normal 11-point document template produces more than 100 logical pages.

The correct execution order is **audit** → **parallel CI foundation** → **Core ISO** → **install** → **recovery** → **guide** → **hardware** → **profiles** → **Windows-app experience** → **commercial hardening**. Development work may run in parallel, but release promotion remains gated by dependencies. Each milestone must leave a bootable or testable artifact.

Executive conclusion

Rain OS should remain a reuse-first Arch integration distribution. It should not merge entire EndeavourOS, CachyOS, and Athena repositories into one package universe. The public repository already contains the beginnings of this approach: an Archiso profile, a KDE package baseline, Rain PKGBUILDS, Windows/WSL helpers, CI, a control center, first-run utility, guide, recovery utility, update preflight, diagrams, and design documents.

The largest corrections required before commercial v1 are repository correctness, signed and reproducible Rain packages, an actual installer integration, a real self-updating guide bundle, hardware-aware profile behavior, tested rollback, a truthful Windows-app workflow, desktop environment support boundaries, accessibility, privacy, security review, and release operations.

Rain OS Core should ship one officially supported desktop first, preferably KDE Plasma, with generic and LTS kernels, a conservative package set, a recovery path, and Urban Rain/Rural Rain themes. GNOME, XFCE, Cinnamon, and further desktops should be profiles with separate test ownership. Windows applications should be offered through a compatibility center based on Wine, Bottles, Lutris, Steam/Proton, Flatpak, and an explicit VM escape hatch.

Current repository snapshot

The public repository contains 138 tracked working-tree files in the shallow audit. Its local syntax and specification tests passed, but that is not equivalent to a successful Archiso build. The build scripts currently require Arch/WSL2 privileges and dependencies that are not available in this Ubuntu audit environment. The build path is authoritative in GitHub Actions for clean, repeatable artifacts. The user's Windows WSL2 environment remains the fast local development and troubleshooting fallback, and real hardware testing remains outside GitHub Actions.

Table of contents

1. [Executive decision and product boundary](#)
2. [Current public repository audit](#)
3. [Prototype readiness and blockers](#)
4. [Commercial release definitions](#)
5. [User demand synthesis](#)
6. [Personas and jobs to be done](#)
7. [Rain OS principles and non-goals](#)

8. [Governance and ownership](#)
9. [Windows host topology](#)
10. [WSL2 installation and tuning](#)
11. [WSL filesystem and performance rules](#)
12. [AI builder toolchain topology](#)
13. [AI tool permissions and secrets](#)
14. [GSD phase loop](#)
15. [ECC evidence and TDD loop](#)
16. [gstack review gates](#)
17. [taste-skill design review](#)
18. [ponytail minimality review](#)
19. [Headroom and local context policy](#)
20. [Skill discovery and provenance](#)
21. [Ralph bounded iteration](#)
22. [One-loop task state machine](#)
23. [Archiso profile contract](#)
24. [One-shot build command](#)
25. [Build cache and acceleration](#)
26. [Build reproducibility](#)
27. [Build logs and artifacts](#)
28. [Package repository architecture](#)
29. [Package signing and key rotation](#)
30. [SBOM and license inventory](#)
31. [Exact source tree](#)
32. [Exact installed filesystem](#)
33. [Core package baseline](#)
34. [KDE package group](#)
35. [GNOME package group](#)
36. [XFCE and Cinnamon groups](#)
37. [Community desktop groups](#)
38. [Kernel strategy](#)
39. [GPU and graphics stack](#)
40. [Firmware and microcode](#)
41. [Boot modes and fallback](#)
42. [Secure Boot and UKI](#)
43. [Installer architecture](#)
44. [Installer UX and change review](#)
45. [Automatic partitioning](#)
46. [Advanced partitioning](#)
47. [LUKS2 encryption](#)
48. [TPM-assisted unlock](#)
49. [Btrfs subvolumes](#)
50. [Snapshots and rollback](#)
51. [ext4 support](#)
52. [Dual boot safety](#)

53. [NetworkManager architecture](#)
54. [Wi-Fi and captive portals](#)
55. [Bluetooth](#)
56. [Audio and PipeWire](#)
57. [Displays and Wayland](#)
58. [Printers and scanners](#)
59. [Power and battery](#)
60. [Suspend and resume](#)
61. [Thermals and performance](#)
62. [File management](#)
63. [Default applications](#)
64. [Flatpak and app sources](#)
65. [App store design](#)
66. [Windows apps product promise](#)
67. [Wine architecture](#)
68. [Bottles integration](#)
69. [Lutris and gaming](#)
70. [Steam and Proton](#)
71. [ProtonDB compatibility UX](#)
72. [Winetricks and dependencies](#)
73. [Windows app security](#)
74. [Windows app data locations](#)
75. [Windows app support matrix](#)
76. [Windows VM fallback](#)
77. [Distrobox and developer containers](#)
78. [Android and mobile services](#)
79. [Gaming stack](#)
80. [Developer Forge profile](#)
81. [Flow performance profile](#)
82. [Shield security profile](#)
83. [Pocket low-resource profile](#)
84. [Profile conflict rules](#)
85. [Control Center information architecture](#)
86. [First-run onboarding](#)
87. [Rain Learning Hub](#)
88. [Self-updating guide protocol](#)
89. [Guide content schema](#)
90. [Guide personalization](#)
91. [Guide rollback](#)
92. [User home directory](#)
93. [Settings and preferences](#)
94. [Notifications and consent](#)
95. [Accessibility baseline](#)
96. [Localization and RTL](#)
97. [Urban and Rural themes](#)

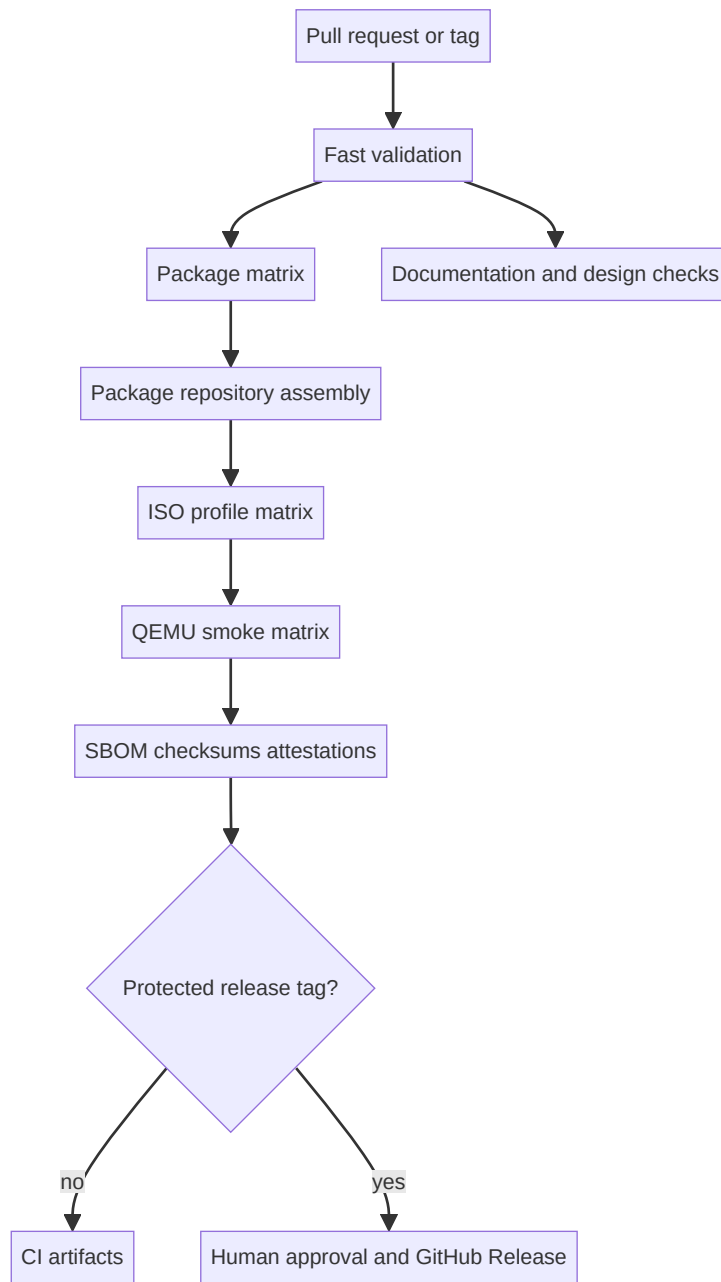
- 98. [Popular environment themes](#)
- 99. [Data and state model](#)
- 100. [Privacy and telemetry](#)
- 101. [Diagnostics bundle](#)
- 102. [Update preflight](#)
- 103. [Update channels](#)
- 104. [Rollback and rescue UI](#)
- 105. [Threat model](#)
- 106. [AppArmor and sandboxing](#)
- 107. [USBGuard and device policy](#)
- 108. [Firewall and network policy](#)
- 109. [Supply-chain security](#)
- 110. [Vulnerability response](#)
- 111. [Hardware certification](#)
- 112. [VM test matrix](#)
- 113. [Installer test matrix](#)
- 114. [Desktop test matrix](#)
- 115. [Windows-app test matrix](#)
- 116. [Accessibility test matrix](#)
- 117. [Performance test matrix](#)
- 118. [Release candidate gate](#)
- 119. [Commercial support model](#)
- 120. [Licensing and trademarks](#)
- 121. [Release channels and updates](#)
- 122. [Documentation and community](#)
- 123. [Roadmap and milestones](#)
- 124. [Risk register](#)
- 125. [Implementation backlog](#)
- 126. [First ISO acceptance checklist](#)
- 127. [Commercial v1 acceptance checklist](#)
- 128. [Final architecture reference](#)

GitHub Actions is the primary build authority

GitHub Actions is the canonical clean build system for Rain OS. Local Windows/WSL2 builds are developer accelerators and debugging tools. A release ISO must be reproducible from a tagged commit in GitHub Actions, with a resolved package manifest, checksums, SBOM, build logs, test reports, and artifact provenance.

The current public workflow is a useful baseline but is sequential. The updated design splits it into independent validation, package, profile, ISO, QEMU-smoke, provenance, and release jobs. Matrix jobs are used only where outputs are genuinely independent. A matrix must not create unsupported desktop or hardware combinations merely because the platform can create them.

Canonical workflow graph



Workflow files

The repository should contain the following workflows:

File	Trigger	Purpose	Release authority
<code>.github/workflows/validate.yml</code>	pull request, push	Shell, Python, JSON, Markdown, PKGBUILD and policy checks	Required status check
<code>.github/workflows/packages.yml</code>	pull request, push, tag	Build Rain PKGBUILDS in a package matrix	Produces signed/unsigned test package artifacts
<code>.github/workflows/iso.yml</code>	workflow call, push, tag	Assemble Archiso profiles after package repository artifact exists	Produces ISO artifacts
<code>.github/workflows/qa.yml</code>	workflow call	QEMU boot/install/smoke tests for each ISO variant	Required before promotion
<code>.github/workflows/nightly.yml</code>	scheduled	Rebuild against current Arch packages and report drift	No automatic stable release
<code>.github/workflows/release.yml</code>	protected tag <code>v*</code>	Attest, sign, publish checksums/SBOM/ISO to GitHub Release	Human-approved

Matrix design

The initial matrix must stay small:

```
strategy:
  fail-fast: false
  matrix:
    package_group: [branding, first-run, learning, recovery, update-preflight, control-center]
```

The first ISO matrix should contain only `core-kde`. Add `core-gnome`, `core-xfce`, or `core-cinnamon` only after each profile has an owner and test evidence:

```
strategy:
  fail-fast: false
  matrix:
    include:
      - profile: core-kde
        desktop: kde
        support: official
      - profile: core-gnome
        desktop: gnome
        support: beta
```

The matrix must not multiply across package group, desktop, kernel, filesystem, GPU, and architecture without a declared test purpose. GitHub Actions imposes a maximum of 256 jobs per matrix, but Rain OS should remain far below that limit for cost and diagnosis clarity [31](#).

Reusable workflows

Package and ISO workflows should be reusable with `workflow_call`, so pull requests, nightly builds, and release tags share the same build logic. Inputs must include `rain_ref`, `profile`, `channel`, `publish_artifacts`, and `sign_release`. Outputs must expose artifact names, checksums, package manifests, and test result paths.

Artifact contract

Every matrix job uploads an artifact with a stable name containing the commit and matrix identity. Package jobs upload packages, repository database files, package manifests, and build logs. ISO jobs upload ISO, checksum, package manifest, SBOM, build metadata, and mkarchiso logs. QA jobs upload screenshots, serial logs, QEMU exit status, installer logs, and test summaries. Artifacts have retention appropriate to their channel; release artifacts are copied to a GitHub Release and long-term archival store.

Cache contract

Use `actions/cache` only for reproducible accelerators such as downloaded Arch packages, language dependencies, and tool downloads. Cache keys must include the architecture, Archiso version, package repository state, profile hash, and lock/revision inputs. Never use a cache as the source of truth for a release package. A cache hit must be safe to delete and rebuild. Cache poisoning is treated as a supply-chain risk.

Concurrency contract

Pull request and branch workflows use a concurrency group based on workflow name and branch/ref and cancel superseded runs. Release tags and signed release jobs use a unique immutable group and must never be cancelled after approval. A scheduled nightly build may be cancelled when a newer nightly run starts, but its failure report must remain visible.

Example:

```
concurrency:
  group: rain-${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: ${{ !startsWith(github.ref, 'refs/tags/v') }}
```

Permissions and trust boundaries

Workflow permissions default to read-only. The build job receives `contents: read` and `actions: read`. The attestation job receives only `id-token: write` and `attestations: write`. The release job receives `contents: write` only after protected-environment approval. Package and ISO build jobs never receive release secrets. Third-party actions are pinned to full commit SHAs and reviewed on update.

Hosted and self-hosted runners

The project must support two runner classes. Hosted runners are preferred for validation, package compilation, documentation, and ordinary QEMU tests. ISO builds that require privileged loop devices, `/dev` access, or nested virtualization must run in an explicitly tested Arch container or an ephemeral self-hosted Linux runner. A self-hosted runner must be dedicated or ephemeral, automatically cleaned after each job, patched, access-restricted, and forbidden from storing signing keys in its workspace.

The current workflow uses an Arch container with `--privileged` and `/dev` exposure. This is a prototype path, not an unconditional production guarantee. The release readiness gate must prove that the selected runner class can mount, squash, boot-test, and clean up correctly. If hosted container privileges are unavailable, the workflow must fail clearly and route to the approved self-hosted runner label rather than silently skipping ISO creation.

Signing and attestations

Build jobs never access private signing keys. A release promotion job in a protected GitHub Environment performs package repository signing and ISO signing only after QA passes. Prefer short-lived OIDC-backed artifact attestations where available. Store public keys in a versioned keyring package and publish checksums, SBOM, source revision, workflow run, and attestation identifiers together. Artifact attestations establish build provenance and integrity claims but do not replace package signature verification [32](#).

Release promotion

A tag does not immediately publish an ISO. The release workflow downloads the exact QA-approved artifacts, verifies checksums, verifies attestations, verifies the source tag, generates release notes and known issues, waits for environment approval, and then publishes the ISO, checksum, SBOM, signatures, provenance statement, installation guide, and recovery guide. Failed promotion leaves artifacts available for diagnosis but does not create a stable release.

Pull-request behavior

Every pull request runs fast validation, syntax tests, changed-package builds, documentation link/schema checks, and a small Archiso profile sanity check where practical. Full ISO and QEMU installation tests run on changes to `archiso/`, `packages/`, `repository/`, installer files, boot files, kernel policy, or release workflows. Design-only changes run accessibility and asset checks without rebuilding every ISO variant.

Parallel AI-agent integration

AI agents may work in parallel on independent issues, but GitHub Actions remains the source of truth. Each agent receives a bounded worktree or branch and must attach a plan, changed-file list, tests, and rollback. ECC/GSD handles planning and evidence, gstack reviews cross-functional impact, taste-skill reviews UI, ponytail reviews unnecessary complexity, Headroom is optional local context

compression, and Ralph is limited to a small issue with a maximum iteration count. No agent may approve a release, access signing secrets, or merge a failing workflow.

GitHub Actions acceptance gates

The parallel build system is accepted when: two independent runs from the same commit produce equivalent package and ISO manifests; matrix failures are isolated and diagnosable; package artifacts can be assembled without rebuilding successful matrix jobs; ISO jobs consume exact upstream artifacts; QEMU smoke tests boot the produced ISO; release jobs can verify checksums and attestations; cancelled branch runs do not cancel release runs; and the Windows WSL2 local fallback can reproduce the same profile and package manifest.

Global acceptance rule

A feature is not complete because an agent generated code or because a package installed in a test environment. It is complete only when the relevant implementation exists, its source and license are recorded, the expected state is observable, the failure path is tested, the rollback or removal path is documented, and a human maintainer accepts the evidence. `newpage`

Page 01: Executive decision and product boundary

Requirement

This page defines **Executive decision and product boundary** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 02: Current public repository audit

Requirement

The repository audit found the following concrete areas: `.gitignore`, `CODE_OF_CONDUCT.md`, `CONTRIBUTING.md`, `LICENSE`, `Makefile`, `README.md`, `SECURITY.md`, `SKILL.md`, `archiso/bootstrap_packages.x86_64`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `archiso/profiledef.sh`, `architecture/BUILD_PIPELINE.mmd`, `architecture/SYSTEM_ARCHITECTURE.mmd`, `branding/rain-logo.jpg`, `branding/rain-logo.png`, `branding/rain-umbrella.svg`, `branding/rain-wallpaper-1080p.jpg`, `branding/rain-wallpaper.jpg`, `build/BUILD_README.md`, `decisions/0001-profile-driven-design.md`, `decisions/PROVENANCE_AND_LEGAL.md`, `design/DESIGN_SYSTEM.md`, `docs/APP_FLOW.md`, `docs/CROSSCHECK_AND_GAPS.md`, `docs/DATA_MODEL.md`, `docs/ENGINEERING_TOOLCHAIN.md`, `docs/ENVIRONMENTS.md`, `docs/IMPLEMENTATION_PLAN.md`, `docs/LOOP_ENGINEERING.md`, `docs/MIGRATION_PLAN.md`, `docs/PRD.md` and additional files under the package, app, architecture, and documentation trees. The public README claims WSL2/Windows orchestration, GitHub Actions ISO builds, Core/Flow/Forge/Shield profiles, and a GPLv3 project license. The local syntax test passed and the specification validator passed.

The audit also found implementation gaps that must be treated as blockers: several PKGBUILDs use `SKIP` checksums; package repository construction tolerates build failures with `|| true`; the Windows script can build from `/mnt/<drive>` instead of the faster WSL filesystem; the live package list includes `apparmor`, `firejail`, `snapper`, and `archinstall` without proving they are configured and safe; the installer launcher references paths that are not yet a complete Rain installer product; and the guide package is not yet the signed state-aware updater described by the specification.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 03: Prototype readiness and blockers

Requirement

This page defines **Prototype readiness and blockers** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an

unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 04: Commercial release definitions

Requirement

This page defines **Commercial release definitions** as an implementation concern rather than a marketing phrase. Commercial readiness requires ownership, release gates, support response, package provenance, SBOMs, documented limitations, and a sustainable update process. It does not require proprietary lock-in or mandatory telemetry.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 05: User demand synthesis

Requirement

This page defines **User demand synthesis** as an implementation concern rather than a marketing phrase. The user experience must expose a single recommended path while preserving terminal control. Every graphical action states what changes, why it changes,

which command or package performs it, and how to undo it. The first release must optimize for learnability and recovery rather than option count.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh` . The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 06: Personas and jobs to be done

Requirement

This page defines **Personas and jobs to be done** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 07: Rain OS principles and non-goals

Requirement

This page defines **Rain OS principles and non-goals** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 08: Governance and ownership

Requirement

This page defines **Governance and ownership** as an implementation concern rather than a marketing phrase. Commercial readiness requires ownership, release gates, support response, package provenance, SBOMs, documented limitations, and a sustainable update process. It does not require proprietary lock-in or mandatory telemetry.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 09: Windows host topology

Requirement

This page defines **Windows host topology** as an implementation concern rather than a marketing phrase. Windows compatibility is a compatibility service, not a promise that every Windows program works. Rain OS should detect the application class, recommend the least complex compatible path, show known limitations, and preserve a Windows VM fallback for software that depends on kernel drivers, anti-cheat, DRM, or unsupported APIs.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 10: WSL2 installation and tuning

Requirement

This page defines **WSL2 installation and tuning** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 11: WSL filesystem and performance rules

Requirement

This page defines **WSL filesystem and performance rules** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 12: AI builder toolchain topology

Requirement

This page defines **AI builder toolchain topology** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 13: AI tool permissions and secrets

Requirement

This page defines **AI tool permissions and secrets** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 14: GSD phase loop

Requirement

This page defines **GSD phase loop** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 15: ECC evidence and TDD loop

Requirement

This page defines **ECC evidence and TDD loop** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 16: gstack review gates

Requirement

This page defines **gstack review gates** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 17: taste-skill design review

Requirement

This page defines **taste-skill design review** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 18: ponytail minimality review

Requirement

This page defines **ponytail minimality review** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 19: Headroom and local context policy

Requirement

This page defines **Headroom and local context policy** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 20: Skill discovery and provenance

Requirement

This page defines **Skill discovery and provenance** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 21: Ralph bounded iteration

Requirement

This page defines **Ralph bounded iteration** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 22: One-loop task state machine

Requirement

This page defines **One-loop task state machine** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 23: Archiso profile contract

Requirement

This page defines **Archiso profile contract** as an implementation concern rather than a marketing phrase. The user experience must expose a single recommended path while preserving terminal control. Every graphical action states what changes, why it changes, which command or package performs it, and how to undo it. The first release must optimize for learnability and recovery rather than option count.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 24: One-shot build command

Requirement

The canonical command must be `./scripts/build-one-shot.sh` inside Arch WSL2. The PowerShell wrapper must call WSL with a Linux path under `/home`, not translate a Windows repository path to `/mnt/c`. The script must update or verify the Arch keyring, install `archiso`, copy the current `reLeng` profile, overlay Rain files, build Rain packages in a non-root `makepkg` context, create a signed local repository, invoke `mkarchiso`, run QEMU smoke tests where possible, and write ISO, checksum, SBOM, package manifest, build log, and source revision metadata.

The command is “one shot” in the sense that it orchestrates the complete artifact pipeline. It cannot truthfully guarantee hardware compatibility without the VM and hardware gates described later.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 25: Build cache and acceleration

Requirement

This page defines **Build cache and acceleration** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 26: Build reproducibility

Requirement

This page defines **Build reproducibility** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 27: Build logs and artifacts

Requirement

This page defines **Build logs and artifacts** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 28: Package repository architecture

Requirement

This page defines **Package repository architecture** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 29: Package signing and key rotation

Requirement

This page defines **Package signing and key rotation** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 30: SBOM and license inventory

Requirement

This page defines **SBOM and license inventory** as an implementation concern rather than a marketing phrase. Commercial readiness requires ownership, release gates, support response, package provenance, SBOMs, documented limitations, and a sustainable update process. It does not require proprietary lock-in or mandatory telemetry.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 31: Exact source tree

Requirement

This page defines **Exact source tree** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 32: Exact installed filesystem

Requirement

This page defines **Exact installed filesystem** as an implementation concern rather than a marketing phrase. The user experience must expose a single recommended path while preserving terminal control. Every graphical action states what changes, why it changes, which command or package performs it, and how to undo it. The first release must optimize for learnability and recovery rather than option count.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh` . The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

Page 33: Core package baseline

Requirement

This page defines **Core package baseline** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 34: KDE package group

Requirement

This page defines **KDE package group** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 35: GNOME package group

Requirement

This page defines **GNOME package group** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 36: XFCE and Cinnamon groups

Requirement

This page defines **XFCE and Cinnamon groups** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 37: Community desktop groups

Requirement

This page defines **Community desktop groups** as an implementation concern rather than a marketing phrase. The user experience must expose a single recommended path while preserving terminal control. Every graphical action states what changes, why it changes, which command or package performs it, and how to undo it. The first release must optimize for learnability and recovery rather than option count.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 38: Kernel strategy

Requirement

This page defines **Kernel strategy** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 39: GPU and graphics stack

Requirement

This page defines **GPU and graphics stack** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 40: Firmware and microcode

Requirement

This page defines **Firmware and microcode** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 41: Boot modes and fallback

Requirement

This page defines **Boot modes and fallback** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 42: Secure Boot and UKI

Requirement

This page defines **Secure Boot and UKI** as an implementation concern rather than a marketing phrase. Security controls must be honest about their state. Rain OS must distinguish active, installed-but-inactive, available, partial, unsupported, and not-applicable protections. No security screen may imply that installing a tool creates a security guarantee.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 43: Installer architecture

Requirement

This page defines **Installer architecture** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 44: Installer UX and change review

Requirement

This page defines **Installer UX and change review** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 45: Automatic partitioning

Requirement

This page defines **Automatic partitioning** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 46: Advanced partitioning

Requirement

This page defines **Advanced partitioning** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh` . The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 47: LUKS2 encryption

Requirement

This page defines **LUKS2 encryption** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 48: TPM-assisted unlock

Requirement

This page defines **TPM-assisted unlock** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh` . The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 49: Btrfs subvolumes

Requirement

This page defines **Btrfs subvolumes** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 50: Snapshots and rollback

Requirement

This page defines **Snapshots and rollback** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 51: ext4 support

Requirement

This page defines **ext4 support** as an implementation concern rather than a marketing phrase. Commercial readiness requires ownership, release gates, support response, package provenance, SBOMs, documented limitations, and a sustainable update process. It does not require proprietary lock-in or mandatory telemetry.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 52: Dual boot safety

Requirement

This page defines **Dual boot safety** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 53: NetworkManager architecture

Requirement

This page defines **NetworkManager architecture** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 54: Wi-Fi and captive portals

Requirement

This page defines **Wi-Fi and captive portals** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 55: Bluetooth

Requirement

This page defines **Bluetooth** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh` . The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 56: Audio and PipeWire

Requirement

This page defines **Audio and PipeWire** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 57: Displays and Wayland

Requirement

This page defines **Displays and Wayland** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 58: Printers and scanners

Requirement

This page defines **Printers and scanners** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 59: Power and battery

Requirement

This page defines **Power and battery** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh` . The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 60: Suspend and resume

Requirement

This page defines **Suspend and resume** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 61: Thermals and performance

Requirement

This page defines **Thermals and performance** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 62: File management

Requirement

This page defines **File management** as an implementation concern rather than a marketing phrase. The user experience must expose a single recommended path while preserving terminal control. Every graphical action states what changes, why it changes, which command or package performs it, and how to undo it. The first release must optimize for learnability and recovery rather than option count.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 63: Default applications

Requirement

This page defines **Default applications** as an implementation concern rather than a marketing phrase. Windows compatibility is a compatibility service, not a promise that every Windows program works. Rain OS should detect the application class, recommend the least complex compatible path, show known limitations, and preserve a Windows VM fallback for software that depends on kernel drivers, anti-cheat, DRM, or unsupported APIs.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 64: Flatpak and app sources

Requirement

This page defines **Flatpak and app sources** as an implementation concern rather than a marketing phrase. Windows compatibility is a compatibility service, not a promise that every Windows program works. Rain OS should detect the application class, recommend the least complex compatible path, show known limitations, and preserve a Windows VM fallback for software that depends on kernel drivers, anti-cheat, DRM, or unsupported APIs.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh` . The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 65: App store design

Requirement

This page defines **App store design** as an implementation concern rather than a marketing phrase. Windows compatibility is a compatibility service, not a promise that every Windows program works. Rain OS should detect the application class, recommend the least complex compatible path, show known limitations, and preserve a Windows VM fallback for software that depends on kernel drivers, anti-cheat, DRM, or unsupported APIs.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 66: Windows apps product promise

Requirement

Rain OS will provide a **Windows App Bridge** in the Control Center. It will classify a requested application as Linux-native, Flatpak, Steam/Proton, Wine/Bottles, Lutris, web/PWA, remote/VM, or unsupported. The user sees the path, isolation boundary, file-access permissions, known compatibility data, and removal procedure before installation.

Rain OS must not claim that Wine is an emulator or that every Windows executable will run. Applications that require kernel drivers, invasive anti-cheat, proprietary device filters, Microsoft Store services, hardware DRM, or unsupported DirectX behavior may fail. A managed Windows VM or dual-boot remains the honest fallback.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 67: Wine architecture

Requirement

This page defines **Wine architecture** as an implementation concern rather than a marketing phrase. Windows compatibility is a compatibility service, not a promise that every Windows program works. Rain OS should detect the application class, recommend the least complex compatible path, show known limitations, and preserve a Windows VM fallback for software that depends on kernel drivers, anti-cheat, DRM, or unsupported APIs.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh` . The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 68: Bottles integration

Requirement

This page defines **Bottles integration** as an implementation concern rather than a marketing phrase. Windows compatibility is a compatibility service, not a promise that every Windows program works. Rain OS should detect the application class, recommend the least complex compatible path, show known limitations, and preserve a Windows VM fallback for software that depends on kernel drivers, anti-cheat, DRM, or unsupported APIs.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh` . The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 69: Lutris and gaming

Requirement

This page defines **Lutris and gaming** as an implementation concern rather than a marketing phrase. Windows compatibility is a compatibility service, not a promise that every Windows program works. Rain OS should detect the application class, recommend the least complex compatible path, show known limitations, and preserve a Windows VM fallback for software that depends on kernel drivers, anti-cheat, DRM, or unsupported APIs.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 70: Steam and Proton

Requirement

This page defines **Steam and Proton** as an implementation concern rather than a marketing phrase. Windows compatibility is a compatibility service, not a promise that every Windows program works. Rain OS should detect the application class, recommend the least complex compatible path, show known limitations, and preserve a Windows VM fallback for software that depends on kernel drivers, anti-cheat, DRM, or unsupported APIs.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 71: ProtonDB compatibility UX

Requirement

This page defines **ProtonDB compatibility UX** as an implementation concern rather than a marketing phrase. Windows compatibility is a compatibility service, not a promise that every Windows program works. Rain OS should detect the application class, recommend the least complex compatible path, show known limitations, and preserve a Windows VM fallback for software that depends on kernel drivers, anti-cheat, DRM, or unsupported APIs.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 72: Winetricks and dependencies

Requirement

This page defines **Winetricks and dependencies** as an implementation concern rather than a marketing phrase. Windows compatibility is a compatibility service, not a promise that every Windows program works. Rain OS should detect the application class, recommend the least complex compatible path, show known limitations, and preserve a Windows VM fallback for software that depends on kernel drivers, anti-cheat, DRM, or unsupported APIs.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 73: Windows app security

Requirement

This page defines **Windows app security** as an implementation concern rather than a marketing phrase. Windows compatibility is a compatibility service, not a promise that every Windows program works. Rain OS should detect the application class, recommend the least complex compatible path, show known limitations, and preserve a Windows VM fallback for software that depends on kernel drivers, anti-cheat, DRM, or unsupported APIs.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 74: Windows app data locations

Requirement

This page defines **Windows app data locations** as an implementation concern rather than a marketing phrase. Windows compatibility is a compatibility service, not a promise that every Windows program works. Rain OS should detect the application class, recommend the least complex compatible path, show known limitations, and preserve a Windows VM fallback for software that depends on kernel drivers, anti-cheat, DRM, or unsupported APIs.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh` . The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 75: Windows app support matrix

Requirement

This page defines **Windows app support matrix** as an implementation concern rather than a marketing phrase. Windows compatibility is a compatibility service, not a promise that every Windows program works. Rain OS should detect the application class, recommend the least complex compatible path, show known limitations, and preserve a Windows VM fallback for software that depends on kernel drivers, anti-cheat, DRM, or unsupported APIs.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 76: Windows VM fallback

Requirement

This page defines **Windows VM fallback** as an implementation concern rather than a marketing phrase. Windows compatibility is a compatibility service, not a promise that every Windows program works. Rain OS should detect the application class, recommend the least complex compatible path, show known limitations, and preserve a Windows VM fallback for software that depends on kernel drivers, anti-cheat, DRM, or unsupported APIs.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 77: Distrobox and developer containers

Requirement

This page defines **Distrobox and developer containers** as an implementation concern rather than a marketing phrase. Windows compatibility is a compatibility service, not a promise that every Windows program works. Rain OS should detect the application class, recommend the least complex compatible path, show known limitations, and preserve a Windows VM fallback for software that depends on kernel drivers, anti-cheat, DRM, or unsupported APIs.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 78: Android and mobile services

Requirement

This page defines **Android and mobile services** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 79: Gaming stack

Requirement

This page defines **Gaming stack** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 80: Developer Forge profile

Requirement

This page defines **Developer Forge profile** as an implementation concern rather than a marketing phrase. The user experience must expose a single recommended path while preserving terminal control. Every graphical action states what changes, why it changes, which command or package performs it, and how to undo it. The first release must optimize for learnability and recovery rather than option count.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 81: Flow performance profile

Requirement

This page defines **Flow performance profile** as an implementation concern rather than a marketing phrase. The user experience must expose a single recommended path while preserving terminal control. Every graphical action states what changes, why it changes, which command or package performs it, and how to undo it. The first release must optimize for learnability and recovery rather than option count.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 82: Shield security profile

Requirement

This page defines **Shield security profile** as an implementation concern rather than a marketing phrase. Security controls must be honest about their state. Rain OS must distinguish active, installed-but-inactive, available, partial, unsupported, and not-applicable protections. No security screen may imply that installing a tool creates a security guarantee.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 83: Pocket low-resource profile

Requirement

This page defines **Pocket low-resource profile** as an implementation concern rather than a marketing phrase. The user experience must expose a single recommended path while preserving terminal control. Every graphical action states what changes, why it changes, which command or package performs it, and how to undo it. The first release must optimize for learnability and recovery rather than option count.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 84: Profile conflict rules

Requirement

This page defines **Profile conflict rules** as an implementation concern rather than a marketing phrase. The user experience must expose a single recommended path while preserving terminal control. Every graphical action states what changes, why it changes, which command or package performs it, and how to undo it. The first release must optimize for learnability and recovery rather than option count.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 85: Control Center information architecture

Requirement

This page defines **Control Center information architecture** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 86: First-run onboarding

Requirement

This page defines **First-run onboarding** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 87: Rain Learning Hub

Requirement

This page defines **Rain Learning Hub** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 88: Self-updating guide protocol

Requirement

This page defines **Self-updating guide protocol** as an implementation concern rather than a marketing phrase. The user experience must expose a single recommended path while preserving terminal control. Every graphical action states what changes, why it changes, which command or package performs it, and how to undo it. The first release must optimize for learnability and recovery rather than option count.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 89: Guide content schema

Requirement

This page defines **Guide content schema** as an implementation concern rather than a marketing phrase. The user experience must expose a single recommended path while preserving terminal control. Every graphical action states what changes, why it changes, which command or package performs it, and how to undo it. The first release must optimize for learnability and recovery rather than option count.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 90: Guide personalization

Requirement

This page defines **Guide personalization** as an implementation concern rather than a marketing phrase. The user experience must expose a single recommended path while preserving terminal control. Every graphical action states what changes, why it changes, which command or package performs it, and how to undo it. The first release must optimize for learnability and recovery rather than option count.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 91: Guide rollback

Requirement

This page defines **Guide rollback** as an implementation concern rather than a marketing phrase. The user experience must expose a single recommended path while preserving terminal control. Every graphical action states what changes, why it changes, which command or package performs it, and how to undo it. The first release must optimize for learnability and recovery rather than option count.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 92: User home directory

Requirement

This page defines **User home directory** as an implementation concern rather than a marketing phrase. The user experience must expose a single recommended path while preserving terminal control. Every graphical action states what changes, why it changes, which command or package performs it, and how to undo it. The first release must optimize for learnability and recovery rather than option count.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 93: Settings and preferences

Requirement

This page defines **Settings and preferences** as an implementation concern rather than a marketing phrase. The user experience must expose a single recommended path while preserving terminal control. Every graphical action states what changes, why it changes, which command or package performs it, and how to undo it. The first release must optimize for learnability and recovery rather than option count.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 94: Notifications and consent

Requirement

This page defines **Notifications and consent** as an implementation concern rather than a marketing phrase. The user experience must expose a single recommended path while preserving terminal control. Every graphical action states what changes, why it changes, which command or package performs it, and how to undo it. The first release must optimize for learnability and recovery rather than option count.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 95: Accessibility baseline

Requirement

This page defines **Accessibility baseline** as an implementation concern rather than a marketing phrase. The user experience must expose a single recommended path while preserving terminal control. Every graphical action states what changes, why it changes, which command or package performs it, and how to undo it. The first release must optimize for learnability and recovery rather than option count.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 96: Localization and RTL

Requirement

This page defines **Localization and RTL** as an implementation concern rather than a marketing phrase. The user experience must expose a single recommended path while preserving terminal control. Every graphical action states what changes, why it changes, which command or package performs it, and how to undo it. The first release must optimize for learnability and recovery rather than option count.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 97: Urban and Rural themes

Requirement

This page defines **Urban and Rural themes** as an implementation concern rather than a marketing phrase. The user experience must expose a single recommended path while preserving terminal control. Every graphical action states what changes, why it changes, which command or package performs it, and how to undo it. The first release must optimize for learnability and recovery rather than option count.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 98: Popular environment themes

Requirement

This page defines **Popular environment themes** as an implementation concern rather than a marketing phrase. The user experience must expose a single recommended path while preserving terminal control. Every graphical action states what changes, why it changes, which command or package performs it, and how to undo it. The first release must optimize for learnability and recovery rather than option count.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 99: Data and state model

Requirement

This page defines **Data and state model** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 100: Privacy and telemetry

Requirement

This page defines **Privacy and telemetry** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 101: Diagnostics bundle

Requirement

This page defines **Diagnostics bundle** as an implementation concern rather than a marketing phrase. Security controls must be honest about their state. Rain OS must distinguish active, installed-but-inactive, available, partial, unsupported, and not-applicable protections. No security screen may imply that installing a tool creates a security guarantee.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 102: Update preflight

Requirement

This page defines **Update preflight** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh` . The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 103: Update channels

Requirement

This page defines **Update channels** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 104: Rollback and rescue UI

Requirement

This page defines **Rollback and rescue UI** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 105: Threat model

Requirement

This page defines **Threat model** as an implementation concern rather than a marketing phrase. Security controls must be honest about their state. Rain OS must distinguish active, installed-but-inactive, available, partial, unsupported, and not-applicable protections. No security screen may imply that installing a tool creates a security guarantee.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 106: AppArmor and sandboxing

Requirement

This page defines **AppArmor and sandboxing** as an implementation concern rather than a marketing phrase. Windows compatibility is a compatibility service, not a promise that every Windows program works. Rain OS should detect the application class, recommend the least complex compatible path, show known limitations, and preserve a Windows VM fallback for software that depends on kernel drivers, anti-cheat, DRM, or unsupported APIs.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 107: USBGuard and device policy

Requirement

This page defines **USBGuard and device policy** as an implementation concern rather than a marketing phrase. Security controls must be honest about their state. Rain OS must distinguish active, installed-but-inactive, available, partial, unsupported, and not-applicable protections. No security screen may imply that installing a tool creates a security guarantee.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 108: Firewall and network policy

Requirement

This page defines **Firewall and network policy** as an implementation concern rather than a marketing phrase. Security controls must be honest about their state. Rain OS must distinguish active, installed-but-inactive, available, partial, unsupported, and not-applicable protections. No security screen may imply that installing a tool creates a security guarantee.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 109: Supply-chain security

Requirement

This page defines **Supply-chain security** as an implementation concern rather than a marketing phrase. Security controls must be honest about their state. Rain OS must distinguish active, installed-but-inactive, available, partial, unsupported, and not-applicable protections. No security screen may imply that installing a tool creates a security guarantee.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 110: Vulnerability response

Requirement

This page defines **Vulnerability response** as an implementation concern rather than a marketing phrase. Security controls must be honest about their state. Rain OS must distinguish active, installed-but-inactive, available, partial, unsupported, and not-applicable protections. No security screen may imply that installing a tool creates a security guarantee.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 111: Hardware certification

Requirement

This page defines **Hardware certification** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 112: VM test matrix

Requirement

This page defines **VM test matrix** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 113: Installer test matrix

Requirement

This page defines **Installer test matrix** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 114: Desktop test matrix

Requirement

This page defines **Desktop test matrix** as an implementation concern rather than a marketing phrase. The user experience must expose a single recommended path while preserving terminal control. Every graphical action states what changes, why it changes, which command or package performs it, and how to undo it. The first release must optimize for learnability and recovery rather than option count.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 115: Windows-app test matrix

Requirement

This page defines **Windows-app test matrix** as an implementation concern rather than a marketing phrase. Windows compatibility is a compatibility service, not a promise that every Windows program works. Rain OS should detect the application class, recommend the least complex compatible path, show known limitations, and preserve a Windows VM fallback for software that depends on kernel drivers, anti-cheat, DRM, or unsupported APIs.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 116: Accessibility test matrix

Requirement

This page defines **Accessibility test matrix** as an implementation concern rather than a marketing phrase. The user experience must expose a single recommended path while preserving terminal control. Every graphical action states what changes, why it changes, which command or package performs it, and how to undo it. The first release must optimize for learnability and recovery rather than option count.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 117: Performance test matrix

Requirement

This page defines **Performance test matrix** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 118: Release candidate gate

Requirement

This page defines **Release candidate gate** as an implementation concern rather than a marketing phrase. Commercial readiness requires ownership, release gates, support response, package provenance, SBOMs, documented limitations, and a sustainable update process. It does not require proprietary lock-in or mandatory telemetry.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 119: Commercial support model

Requirement

This page defines **Commercial support model** as an implementation concern rather than a marketing phrase. Commercial readiness requires ownership, release gates, support response, package provenance, SBOMs, documented limitations, and a sustainable update process. It does not require proprietary lock-in or mandatory telemetry.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 120: Licensing and trademarks

Requirement

This page defines **Licensing and trademarks** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 121: Release channels and updates

Requirement

This page defines **Release channels and updates** as an implementation concern rather than a marketing phrase. Commercial readiness requires ownership, release gates, support response, package provenance, SBOMs, documented limitations, and a sustainable update process. It does not require proprietary lock-in or mandatory telemetry.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 122: Documentation and community

Requirement

This page defines **Documentation and community** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 123: Roadmap and milestones

Requirement

This page defines **Roadmap and milestones** as an implementation concern rather than a marketing phrase. Commercial readiness requires ownership, release gates, support response, package provenance, SBOMs, documented limitations, and a sustainable update process. It does not require proprietary lock-in or mandatory telemetry.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 124: Risk register

Requirement

This page defines **Risk register** as an implementation concern rather than a marketing phrase. Commercial readiness requires ownership, release gates, support response, package provenance, SBOMs, documented limitations, and a sustainable update process. It does not require proprietary lock-in or mandatory telemetry.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 125: Implementation backlog

Requirement

This page defines **Implementation backlog** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 126: First ISO acceptance checklist

Requirement

The first ISO is accepted only when it boots in UEFI QEMU, reaches the live desktop, starts the installer path, completes an ext4 installation, completes a Btrfs/LUKS2 installation in a test VM, boots the generic and LTS kernels, connects to a test network, starts audio, opens Rain Guide offline, runs update preflight, exports a secret-scrubbed diagnostic bundle, and documents every known failure.

A successful `mkarchiso` command alone is not acceptance. The artifact must have a checksum and package manifest, and the installation must be repeatable from a clean WSL2 build directory.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 127: Commercial v1 acceptance checklist

Requirement

This page defines **Commercial v1 acceptance checklist** as an implementation concern rather than a marketing phrase. Commercial readiness requires ownership, release gates, support response, package provenance, SBOMs, documented limitations, and a sustainable update process. It does not require proprietary lock-in or mandatory telemetry.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Page 128: Final architecture reference

Requirement

This page defines **Final architecture reference** as an implementation concern rather than a marketing phrase. The build layer must remain a thin, inspectable wrapper over Archiso. It must use the current installed Archiso profile, keep the repository under the WSL filesystem, and refuse unsafe host paths. The implementation artifact is a versioned script with shellcheck coverage, a clean build log, a resolved package list, a checksum, and a source revision record.

The implementation must be expressed as small Rain-owned packages, configuration overlays, or documented upstream selections. It must not create a second package manager, hide the underlying Arch command, silently add privileged services, or make an unmeasured performance claim. When an upstream component already satisfies the requirement, Rain should configure or wrap it and record the upstream version in the provenance ledger.

Implementation contract

Area	Required decision	Initial implementation artifact	Owner	Evidence
Scope	Define one supported path and explicit non-goals	PRD decision record	Product	Approved decision
Code/config	Prefer reuse, then configure, wrap, patch upstream, or replace	Versioned file or PKGBUILD	Engineering	Review diff
Data	Minimize state and document retention	Schema or state file	Platform	Migration test
UX	Show current state, impact, and rollback	Flow and screen contract	Design	Accessibility review
Security	Bound privilege and verify provenance	Policy/test	Security	Threat-model evidence
Release	Make failure observable and reversible	CI gate and runbook	Release	Passing gate

Required files and packages

The first implementation pass should inspect or update: `archiso/profiledef.sh`, `archiso/packages.x86_64`, `archiso/pacman.conf`, `scripts/build-iso.sh`, `scripts/build-windows.ps1`, `scripts/build-windows.bat`, `scripts/validate-spec.sh`, `tests/test-syntax.sh`. The actual list must be refined in the issue before coding; a file listed here is a planned touch point, not evidence that the feature already exists.

Configuration contract

Configuration must be declarative where possible, validated before use, versioned, and safe when absent. Secrets must not be placed in Git, an ISO profile, an AI-agent prompt, or a desktop configuration file. Every configuration change needs a dry-run or preview path and a recovery note.

Verification and failure handling

The task owner must add a focused test, a negative test, and a recovery test. The negative test must cover missing hardware, missing network, outdated guide metadata, invalid package signatures, unsupported application classes, insufficient disk space, and interrupted transactions where relevant. Repeated failures stop an automated loop and create a human diagnosis item.

Exit criteria

This page exits when the implementation artifact exists, the test evidence is attached, the upstream source and license are recorded, the documentation is updated, and the feature can be removed without making Core unbootable.

\newpage

Appendix A: Complete public repository inventory

The following inventory was generated from the shallow clone used for this audit. It is included so that the next implementation loop can map every current file to an owner, test, license record, and disposition.

Path	Current role	Required dispositic
<code>.github/workflows/build-iso.yml</code>	Continuous integration workflow	Pin actions and ac ISO boot/install/recove gates
<code>.github/workflows/validate.yml</code>	Continuous integration workflow	Pin actions and ac ISO boot/install/recove gates
<code>.gitignore</code>	Project governance or root configuration	Review for commercial releas policy
<code>CODE_OF_CONDUCT.md</code>	Project governance or root configuration	Review for commercial releas policy
<code>CONTRIBUTING.md</code>	Project governance or root configuration	Review for commercial releas policy
<code>LICENSE</code>	Project governance or root configuration	Review for commercial releas policy
<code>Makefile</code>	Project governance or root configuration	Review for commercial releas policy
<code>README.md</code>	Project governance or root configuration	Review for commercial releas policy
<code>SECURITY.md</code>	Project governance or root configuration	Review for commercial releas policy
<code>SKILL.md</code>	Project governance or root configuration	Review for commercial releas policy
<code>apps/control-center/README.md</code>	Prototype graphical application or documentation	Add functional and accessibility tests; avoid placeholder actions
<code>apps/control-center/rain_control_center_gui.py</code>	Prototype graphical application or documentation	Add functional and accessibility tests; avoid placeholder actions
<code>apps/first-run/README.md</code>	Prototype graphical	Add functional and accessibility tests;

Path	Current role	Required disposition
	application or documentation	avoid placeholder actions
<code>apps/first-run/rain_first_run_gui.py</code>	Prototype graphical application or documentation	Add functional and accessibility tests; avoid placeholder actions
<code>apps/learning-hub/README.md</code>	Prototype graphical application or documentation	Add functional and accessibility tests; avoid placeholder actions
<code>archiso/airootfs/etc/calamares/branding/rain-os/branding.desc</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/etc/calamares/branding/rain-os/rain-logo.png</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/etc/calamares/modules/packages.conf</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/etc/calamares/modules/welcome.conf</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/etc/calamares/settings.conf</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/etc/hostname</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/etc/issue</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/etc/os-release</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test

Path	Current role	Required dispositic
<code>archiso/airootfs/etc/rain-os/profile.conf</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/etc/rain-os/release.json</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/etc/sddm.conf.d/autologin.conf</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/etc/sudoers.d/g_wheel</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/etc/systemd/system/multi-user.target.wants/display-manager.service</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/etc/systemd/system/multi-user.target.wants/rain-live-setup.service</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/etc/systemd/system/rain-live-setup.service</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/usr/local/bin/rain-control-center</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/usr/local/bin/rain-first-run-gui</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/usr/local/bin/rain-install-launcher</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test

Path	Current role	Required dispositic
<code>archiso/airootfs/usr/local/bin/rain-live-setup</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/usr/local/bin/rain-welcome-launcher</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/usr/share/applications/rain-control-center.desktop</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/usr/share/applications/rain-installer.desktop</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/usr/share/applications/rain-learning-hub.desktop</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/usr/share/applications/rain-recovery.desktop</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/usr/share/applications/rain-welcome.desktop</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/usr/share/icons/hicolor/128x128/apps/rain-os.png</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/usr/share/icons/hicolor/256x256/apps/rain-os.png</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/usr/share/icons/hicolor/32x32/apps/rain-os.png</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test

Path	Current role	Required dispositic
<code>archiso/airootfs/usr/share/icons/hicolor/48x48/apps/rain-os.png</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/usr/share/icons/hicolor/512x512/apps/rain-os.png</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/usr/share/icons/hicolor/64x64/apps/rain-os.png</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/usr/share/pixmaps/rain-os.png</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/usr/share/wallpapers/RainOS/contents/images/1920x1080.jpg</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/airootfs/usr/share/wallpapers/RainOS/metadata.json</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/bootstrap_packages.x86_64</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/efiboot/loader/entries/01-rain-linux.conf</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/efiboot/loader/entries/02-rain-linux-lts.conf</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/efiboot/loader/loader.conf</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test

Path	Current role	Required dispositic
<code>archiso/packages.x86_64</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/pacman.conf</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/profiledef.sh</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/syslinux/archiso.cfg</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>archiso/syslinux/syslinux.cfg</code>	Archiso profile, package, or live-image input	Keep only after current Archiso compatibility test
<code>architecture/BUILD_PIPELINE.mmd</code>	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
<code>architecture/SYSTEM_ARCHITECTURE.mmd</code>	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
<code>architecture/rendered/application-flows.png</code>	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
<code>architecture/rendered/build-pipeline.png</code>	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
<code>architecture/rendered/system-architecture.png</code>	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
<code>branding/rain-logo.jpg</code>	Branding or wallpaper asset	Record copyright and license; test theme packaging
<code>branding/rain-logo.png</code>	Branding or wallpaper	Record copyright and license; test

Path	Current role	Required dispositic
	asset	theme packaging
branding/rain-umbrella.svg	Branding or wallpaper asset	Record copyright and license; test theme packaging
branding/rain-wallpaper-1080p.jpg	Branding or wallpaper asset	Record copyright and license; test theme packaging
branding/rain-wallpaper.jpg	Branding or wallpaper asset	Record copyright and license; test theme packaging
build/BUILD_README.md	Project governance or root configuration	Review for commercial releas policy
decisions/0001-profile-driven-design.md	Project governance or root configuration	Review for commercial releas policy
decisions/PROVENANCE_AND_LEGAL.md	Project governance or root configuration	Review for commercial releas policy
design/DESIGN_SYSTEM.md	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
docs/APP_FLOW.md	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
docs/CROSSCHECK_AND_GAPS.md	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
docs/DATA_MODEL.md	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
docs/ENGINEERING_TOOLCHAIN.md	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
docs/ENVIRONMENTS.md	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
docs/IMPLEMENTATION_PLAN.md	Specification, design,	Map to one releas requirement and

Path	Current role	Required dispositic
	architecture, or QA artifact	maintain with implementation
docs/LOOP_ENGINEERING.md	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
docs/MIGRATION_PLAN.md	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
docs/PRD.md	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
docs/REFERENCE_INDEX.md	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
docs/REUSE_STRATEGY.md	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
docs/SECURITY_MODEL.md	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
docs/SELF_UPDATING_GUIDE.md	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
docs/STARTING_CONCEPT.md	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
docs/TRD.md	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
docs/USER_LEARNING_PATH.md	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
docs/WORKFLOW_GOVERNANCE.md	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
docs/guide-index.example.json	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation

Path	Current role	Required dispositic
docs/guide-index.schema.json	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
manifests/BUILDER_TOOLS.csv	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
manifests/FILES.md	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
manifests/PACKAGES.md	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
manifests/PROVENANCE_LEDGER.csv	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
manifests/UPSTREAM_COMPONENTS.md	Specification, design, architecture, or QA artifact	Map to one releas requirement and maintain with implementation
packages/rain-branding/PKGBUILD	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
packages/rain-branding/icons/128x128/rain-os.png	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
packages/rain-branding/icons/256x256/rain-os.png	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
packages/rain-branding/icons/32x32/rain-os.png	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
packages/rain-branding/icons/48x48/rain-os.png	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
packages/rain-branding/icons/512x512/rain-os.png	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
packages/rain-branding/icons/64x64/rain-os.png	Rain package recipe or	Build in clean makepkg; replace

Path	Current role	Required dispositic
	installed helper	SKIP checksums before release
packages/rain-branding/rain-logo.png	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
packages/rain-branding/rain-rural.colors	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
packages/rain-branding/rain-umbrella.svg	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
packages/rain-branding/rain-urban.colors	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
packages/rain-control-center/PKGBUILD	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
packages/rain-control-center/bin/__pycache__/rain-control-centercpython-312.pyc	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
packages/rain-control-center/bin/rain-control-center	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
packages/rain-control-center/rain-control-center.desktop	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
packages/rain-first-run/PKGBUILD	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
packages/rain-first-run/bin/__pycache__/rain-first-run-guicpython-312.pyc	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
packages/rain-first-run/bin/rain-first-run	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
packages/rain-first-run/bin/rain-first-run-gui	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release

Path	Current role	Required dispositic
<code>packages/rain-first-run/rain-first-run.desktop</code>	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
<code>packages/rain-learning-hub/PKGBUILD</code>	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
<code>packages/rain-learning-hub/bin/rain-guide</code>	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
<code>packages/rain-learning-hub/lessons/01-welcome.md</code>	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
<code>packages/rain-learning-hub/lessons/02-updates.md</code>	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
<code>packages/rain-learning-hub/lessons/03-recovery.md</code>	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
<code>packages/rain-learning-hub/rain-guide.desktop</code>	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
<code>packages/rain-recovery-tools/PKGBUILD</code>	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
<code>packages/rain-recovery-tools/bin/rain-recovery</code>	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
<code>packages/rain-recovery-tools/rain-recovery.desktop</code>	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
<code>packages/rain-update-preflight/PKGBUILD</code>	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
<code>packages/rain-update-preflight/bin/rain-update-preflight</code>	Rain package recipe or installed helper	Build in clean makepkg; replace SKIP checksums before release
<code>packages/rain-update-preflight/rain-update-preflight.desktop</code>	Rain package recipe or	Build in clean makepkg; replace

Path	Current role	Required disposition
	installed helper	SKIP checksums before release
qa/QA_MATRIX.md	Specification, design, architecture, or QA artifact	Map to one release requirement and maintain with implementation
repository/build-repo.sh	Build, repository, or orchestration script	Shellcheck, fail closed, remove tolerated build failures
repository/keys/README.md	Build, repository, or orchestration script	Shellcheck, fail closed, remove tolerated build failures
repository/signing-policy.md	Build, repository, or orchestration script	Shellcheck, fail closed, remove tolerated build failures
scripts/build-iso.sh	Build, repository, or orchestration script	Shellcheck, fail closed, remove tolerated build failures
scripts/build-windows.bat	Build, repository, or orchestration script	Shellcheck, fail closed, remove tolerated build failures
scripts/build-windows.ps1	Build, repository, or orchestration script	Shellcheck, fail closed, remove tolerated build failures
scripts/setup-wsl-admin.bat	Build, repository, or orchestration script	Shellcheck, fail closed, remove tolerated build failures
scripts/validate-spec.sh	Build, repository, or orchestration script	Shellcheck, fail closed, remove tolerated build failures
tests/test-syntax.sh	Project governance or root configuration	Review for commercial release policy

Audit observations

The inventory demonstrates that Rain OS is a prototype repository with meaningful integration work, not a complete distribution. It has a live Archiso profile and several Rain package recipes, but an ISO can only be considered a commercial build after the package repository, installer behavior, guide updater, signatures, hardware tests, and recovery tests are real and reproducible.

The most important corrective actions are to stop tolerating package build failures, replace `SKIP` checksums with generated checksums or an explicit source integrity mechanism, keep the build tree inside WSL, separate the public prototype from release

signing material, test every GUI path against the installed binaries it launches, and introduce a release manifest that ties every package and configuration file to a tested artifact.

\\newpage

Architecture diagrams

Diagram 1: Windows to ISO build topology



Diagram 2: One-shot build pipeline

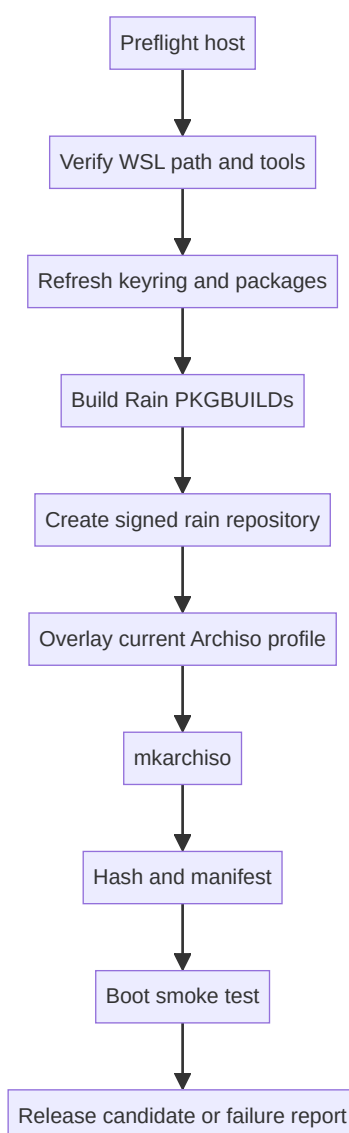


Diagram 3: Rain OS layered architecture

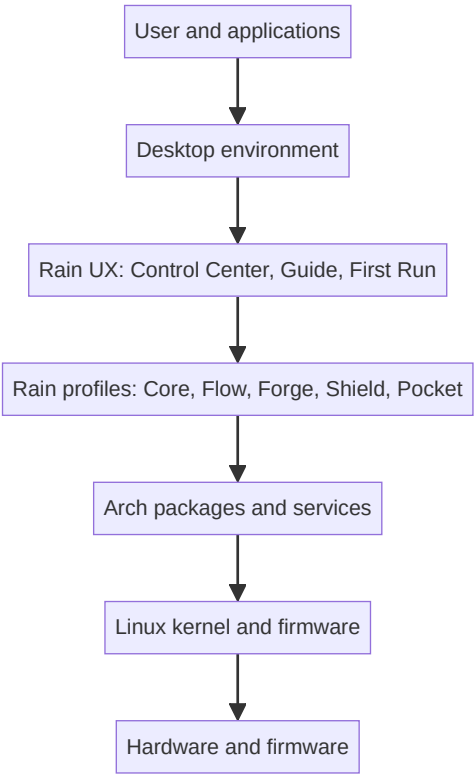


Diagram 4: Installer flow

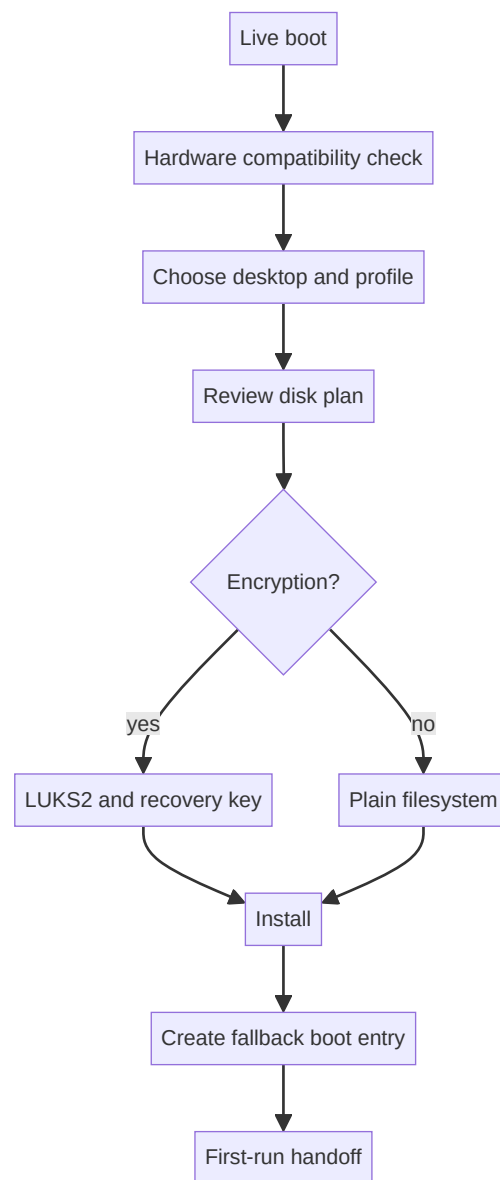


Diagram 5: Update and rollback

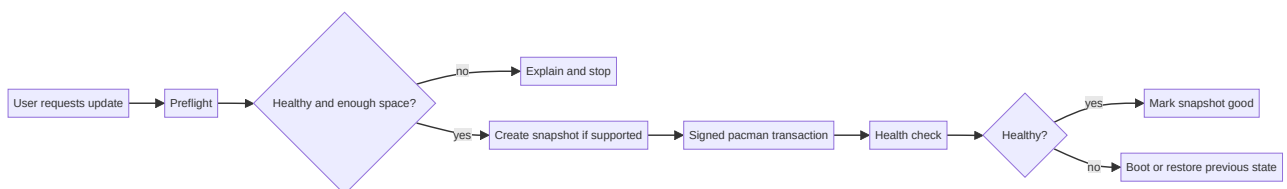


Diagram 6: Self-updating guide

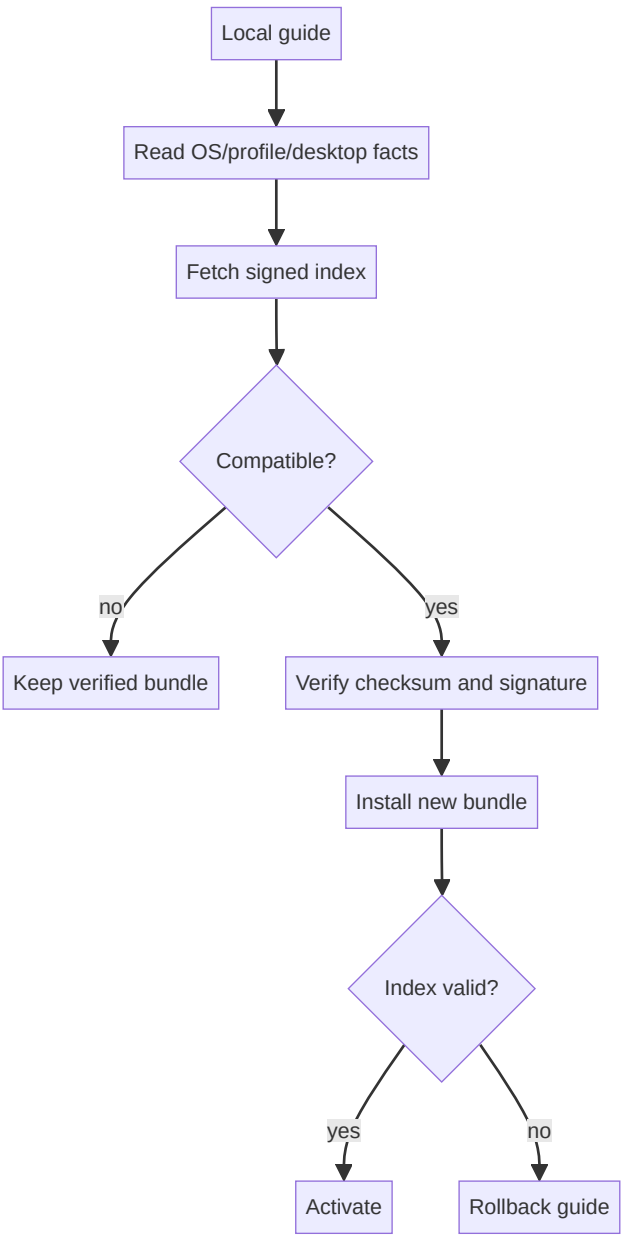


Diagram 7: Windows application decision tree

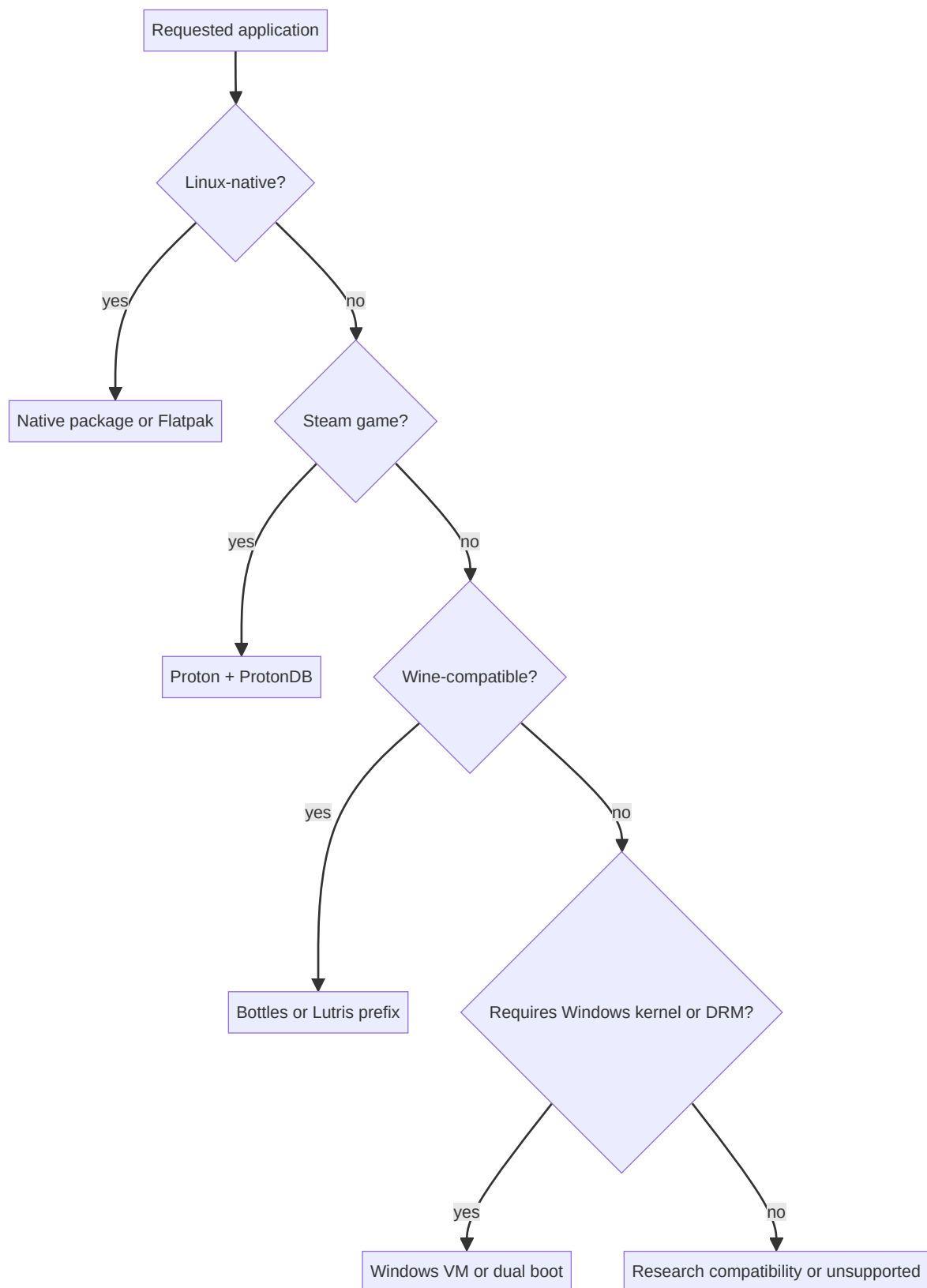


Diagram 8: Application isolation

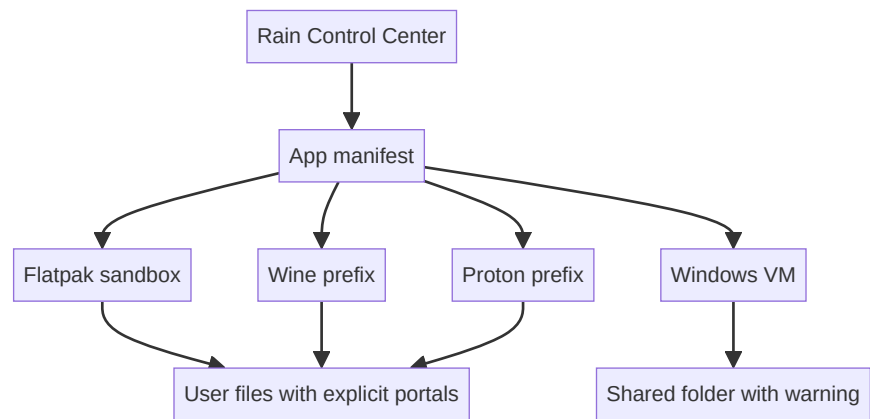


Diagram 9: AI-agent development loop

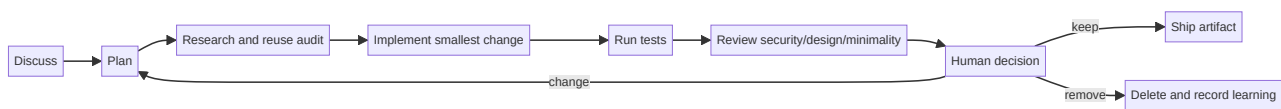


Diagram 10: Profile composition

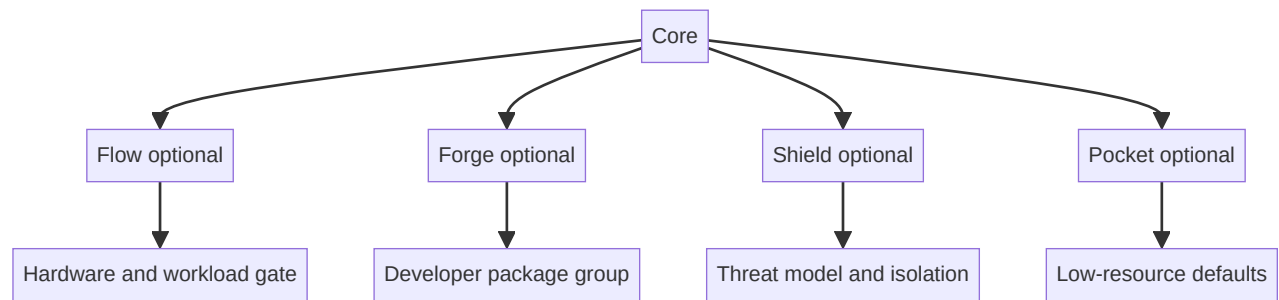


Diagram 11: Package provenance

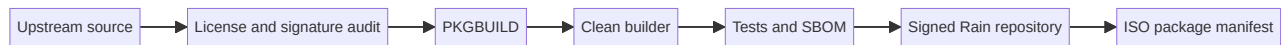


Diagram 12: Recovery layers

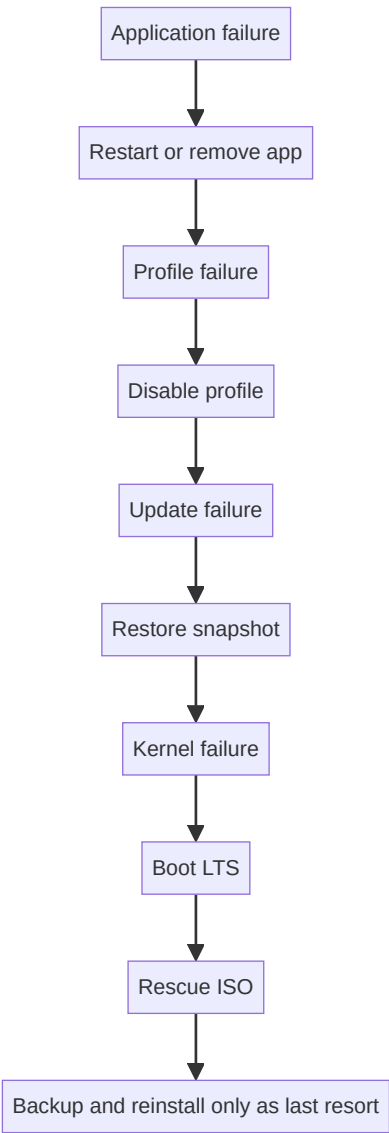


Diagram 13: Support and telemetry boundary

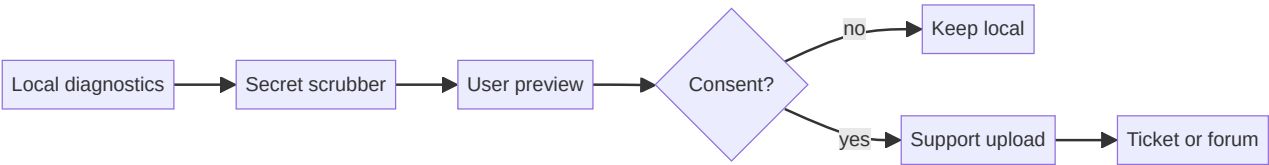


Diagram 14: Release channels

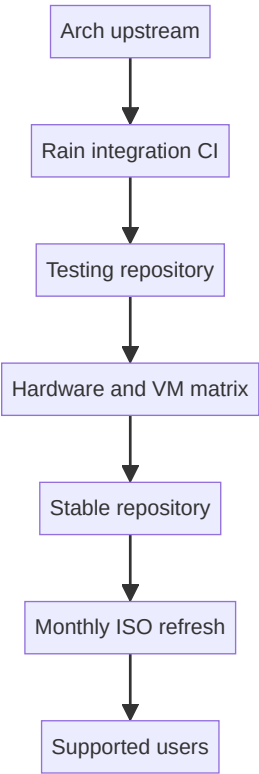


Diagram 15: Commercial support loop

mermaid flowchart LR A[User issue] --> B[Guide and diagnostics] B --> C[Community triage] C --> D[Support escalation] D --> E[Reproducible bug] E --> F[Upstream or Rain fix] F --> G[Regression test] G --> H[Guide update]

\newpage

References